The twisted frame: tracking the measurements of the $\mathbb{D}_4$ code exactly and fast

Study 1 of the $\mathbb{D}_4$ code-switching project (explanatory note)

12 November 2026

Abstract

To estimate the error threshold of the $\mathbb{D}_4$ Clifford-hierarchy stabilizer code we must simulate, many times, how a noisy code state responds to the measurement of its checks. The checks are not Pauli operators, they stop commuting when an error is present, and the quantum state is far too large to store. This note explains, for readers without a background in quantum error correction, the representation we use instead. The state is written as a set of bit strings that solves a system of linear equations modulo 2, carrying a sign given by a quadratic function, on top of a fixed cubic sign that never has to be computed. Every noise event and every measurement keeps this form, so the state can be followed exactly with small binary matrices. We call it the *twisted frame*; the name is ours, not an established term. We explain why it is valid for the memory experiment of study 1 and why it is the right tool there.

The idea in one sentence

However many errors and measurements occur, the state of the $\mathbb{D}_4$ code always has the form

(solutions of a linear system mod 2) $\times$ *(sign of a quadratic function)* $\times$ *(a fixed cubic sign)*,

and only the first two factors ever change.

1 What we need to compute

Study 1 simulates a *memory experiment*. A logical state is encoded in the code, and then, round after round, (1) random errors hit the physical qubits, (2) the checks (stabilizers) of the code are measured to obtain a syndrome, and (3) a decoder infers and corrects the errors. At the end we ask whether the logical information survived. Repeating this 10^5 times for several system sizes L and error rates p gives failure-rate curves whose crossings estimate the threshold. The tool we need answers one question exactly: *for a given realisation of the noise, which measurement outcomes occur, and with what probabilities?*

1.1 Why the standard tools fail

- **Store the state vector.** The $L \times L$ torus has $n = 9L^2$ qubits, i.e. 2^n amplitudes (2^{81} at $L = 3$). Restricting to error-free configurations still leaves $2^{27} \approx 1.3 \times 10^8$ amplitudes at $L = 3$ and 2^{75} at $L = 5$.
- **Stabilizer simulators** such as Stim require every check to be a product of Pauli operators. The X-type checks of the $\mathbb{D}_4$ code are Pauli operators multiplied by controlled-Z (CZ) gates, and measuring them needs CCZ gates.

- **The surrogate of the previous study** replaced the checks by plain Pauli checks and added independent noise for the CZ factors. It cannot reproduce the non-commutativity described next.

1.2 What makes the $\mathbb{D}_4$ code hard

- **The checks carry CZ gates.** The check of colour c at vertex v is $S = X^s(-1)^{q(a)}$: X^s flips six qubits and $(-1)^{q(a)}$ is the sign produced by two CZ gates, so q is a sum of products of two bits (a quadratic function).
- **Errors make the checks non-commuting.** Without errors all checks commute. An X error creates a *flux*, and then some checks of different colours anticommute: measuring a check S_1, then S_2, then S_1 again reproduces the first outcome of S_1 only with probability $1/2$. Outcomes are random, and the syndrome is no longer a fixed function of the errors.
- **Corrected errors can leave a CZ-type residue.** Even after a flux has been corrected, the state can carry a CZ-type error that randomises checks of other colours.

2 Bits, flat configurations and the code state

A computational-basis state $|a\rangle$ is a bit string $a \in \mathrm{GF}(2)^n$. In the $\mathbb{D}_4$ code the qubits sit on the edges of the lattice, three colours per edge, so a assigns a bit to every edge and colour (a *gauge configuration*). A configuration is *flat* if, for every triangle and colour, the three edge bits add up to 0 modulo 2; a triangle where they add up to 1 carries a flux. The Z-type checks measure exactly these sums.

The error-free code state is

$$|\psi_0\rangle = \sum_{b \in C_0} (-1)^{\phi_0(b)} |b\rangle, \tag{1}$$

where C_0 is the set of flat configurations (in the logical sectors we track) and ϕ_0 is a *cubic* function built from products of one bit of each colour. This cubic sign is the "twist" of the twisted gauge theory; the toric code would have no sign at all.

Flatness is a set of linear equations, so C_0 is the solution set of a linear system: every $b \in C_0$ can be written as $b = Mt$ with a short coordinate vector $t \in \mathrm{GF}(2)^k$ (one bit per vertex and colour, plus one winding bit per logical colour; $k = 3L^2$). At $L = 9$, $k = 243$, which is tiny compared with 2^{729} amplitudes.

The fact behind fast Clifford simulators. Every stabilizer state (any state a Clifford circuit can prepare) has the form $\psi(a) \propto i^{\ell(a)}(-1)^{Q(a)}\mathbf{1}[a \in \mathcal{A}]$ with $\mathcal{A}$ an affine subspace and Q quadratic (Dehaene–De Moor). That is why Stim-like simulators are fast. Because of the cubic sign ϕ_0, code states of the $\mathbb{D}_4$ code are *not* stabilizer states. The twisted frame shows that this does not matter.

3 The representation

Proposition (Sec. 4 of the study-1 manuscript)**.** *Starting from $|\psi_0\rangle$ and applying any sequence of Pauli operators, CZ gates, measurements of Z-type checks and projective measurements of the dressed X-type checks (possibly multiplied by a Pauli Z), the state is, up to normalisation,*

$$\psi(a) = (-1)^{\phi_0(b)+Q(b)}\,\mathbf{1}[b \in B], \qquad b = a + x, \tag{2}$$

where $x \in \mathrm{GF}(2)^n$ is the accumulated X frame, $B \subseteq C_0$ is an affine subspace and Q is a quadratic form over $\mathrm{GF}(2)$. Every measurement outcome has probability 0, $\frac{1}{2}$ or 1.

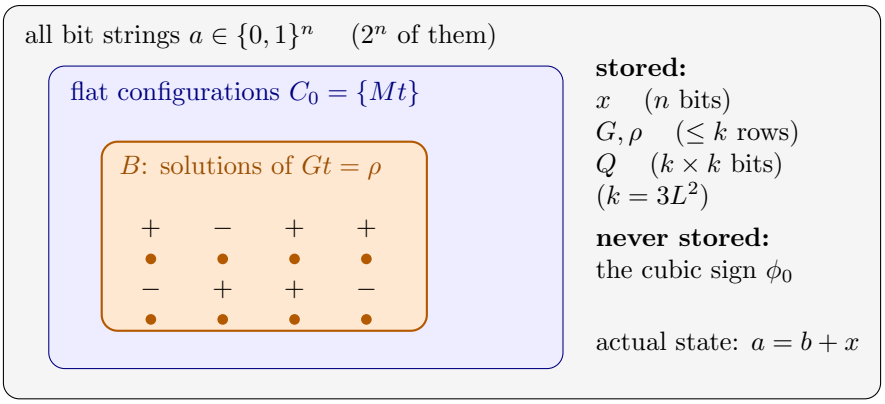


Figure 1: The twisted frame. The amplitude is non-zero only on the set B (orange) of flat configurations, shifted by the X frame x. On B the amplitude is ± 1; the sign is the fixed cubic sign $(-1)^{\phi_0}$ times the quadratic sign $(-1)^Q$ (the $\pm$ marks illustrate the latter). Only x, the linear system $Gt = \rho$ that defines B, and Q are stored.

The three stored ingredients are (Fig. 1):

ingredient	meaning	storage
X frame x	all X errors and X corrections so far; we look at the state "through" this shift	n bits
B	the flat configurations present in the superposition (an affine subspace of C_0)	a row-reduced linear system, at most k rows
Q	signs acquired from noise and measurements; a *quadratic* form	$k \times k$ bits plus a linear part
ϕ_0	the cubic sign of the initial code state	*not stored*

The name: *twisted* refers to the cubic sign ϕ_0 of the twisted gauge theory, *frame* to the Pauli frame x familiar from stabilizer simulators.

Why ϕ_0 is never needed. A check S leaves the code state unchanged, $S|\psi_0\rangle = |\psi_0\rangle$. Written configuration by configuration this means that for every flat b

$$\phi_0(b + s) + \phi_0(b) = q(b + s), \tag{3}$$

where s marks the qubits flipped by the check and q is the quadratic function of its CZ gates. The simulation only ever needs the *ratio* of the amplitudes at b and $b + s$, i.e. a *difference* of ϕ_0, and Eq. (3) replaces that difference by the quadratic q.

4 How each operation updates the state

operation	update	cost
X error or X correction	flip a bit of x	one bit
Z error	add a linear term to Q (multiply by $(-1)^{a_q}$)	one row XOR
CZ (e.g. a circuit fault)	add a quadratic term to Q (multiply by $(-1)^{a_i a_j}$)	two rows and columns
Z-type check	outcome read from x (B is flat, so every flux lives in x)	three bits
dressed X-type check	one of the two cases below	a few matrix rows

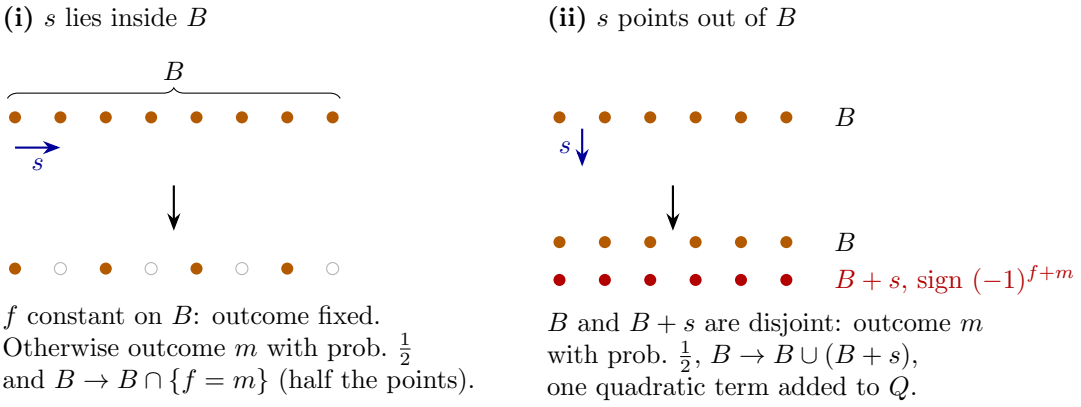


Figure 2: The two cases of a dressed-check measurement, drawn schematically (dots are configurations in B). Case (ii) is exactly the non-commutativity of the checks: an earlier measurement has narrowed B in the direction in which the present check moves configurations.

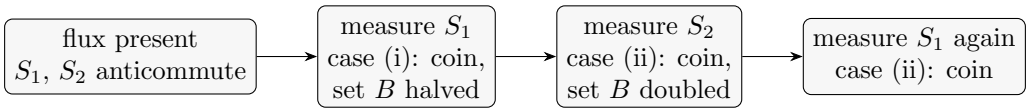


Figure 3: Re-randomisation while a flux survives. The second outcome of S_1 agrees with the first only with probability $\frac{1}{2}$.

4.1 Measuring a dressed check: two cases

To measure $S = X^s(-1)^q$ we first form the *affine* function

$$f(b) = q(b + s + x) + q(b + s) + Q(b + s) + Q(b), \tag{4}$$

the sign of the ratio of $S\psi$ to ψ at b. It is affine because q and Q are quadratic and we take a difference. The part $q(b + s + x) + q(b + s)$ is the Z-type effect produced when the X frame meets a CZ gate (the "Pauli dressing"); CZ-type residues enter here as well.

- **Case (i): s lies in the direction space of B** (Fig. 2, left). If f is constant on B, the outcome is certain. Otherwise the outcome m has probability $\frac{1}{2}$ and B is restricted to $\{f = m\}$, half of its points.
- **Case (ii): s does not lie in the direction space of B** (Fig. 2, right). Then B and $B + s$ are disjoint, the outcome has probability $\frac{1}{2}$, B grows to $B \cup (B + s)$, and the sign $(-1)^{f+m}$ is attached to the new half, which adds one quadratic term to Q.

In both cases the state returns to the form (2), and the outcome probability is 0, $\frac{1}{2}$ or 1.

Non-commutativity is case (ii). Suppose a flux is present and checks S_1 and S_2 anticommute on the state (Fig. 3). Measuring S_1 (case (i)) restricts the set B along the condition of S_1. Measuring S_2 then moves configurations exactly in that direction, so it falls into case (ii): a fair coin, and B is enlarged again. Measuring S_1 once more meets an unrestricted direction and is again a fair coin. "$S_1 \to S_2 \to S_1$ does not reproduce S_1" therefore emerges from the update rules with no special treatment.

4.2 One shot

1. Start: $x = 0$, $B = C_0$, $Q = 0$; an equal superposition of the eight logical (winding) sectors of the three colours.

2. Every round: noise (updates x and Q) $\to$ Z-type checks (read from x) $\to$ real-time flux correction (updates x) $\to$ dressed checks colour by colour (Sec. 4.1).

3. End: close the remaining fluxes, remove x by applying products of checks, and measure one perfect round. Without fluxes all checks commute, so the state is now a code state times charges.

4. Logical fidelity: for each of the eight sectors, read whether it lies in B and the value of Q, and compute the overlap with the initial state as an *exact number*, not a sample.

5 Cost

The linear system for B and the form Q are binary matrices of size at most $k \times k$ ($k = 3L^2$), stored 64 bits to a word. A measurement costs $O(k^2)$ bit operations; at $L = 9$ ($k = 243$) a matrix occupies about 7 kB. Measured cost per shot (phenomenological noise at $p = 2\%$, decoding included, one process):

L	qubits	twisted frame (production)	Pauli proxy (same task)	dense state vector
3	81	0.33 ms	4.1 ms	2^{27} amplitudes; 0.1–1 s per shot on a GPU
5	225	0.89 ms	18 ms	impossible (2^{75})
7	441	2.2 ms	79 ms	impossible
9	729	6.3 ms	236 ms	impossible

The Pauli-proxy column is a per-shot Python implementation; driven by the same per-shot code, it is 3.5–6.8 times slower than the twisted frame, and 37 times slower than the batched production engine at $L = 9$ (the latter ratio also includes implementation differences).

6 How we know it is right

- **Proof.** The Proposition above (proved in the study-1 manuscript).
- **Dense state vector at $L = 3$.** Driven by the same noise, a brute-force simulation that applies every operation literally (in the circuit-level model every CNOT and CCZ) gives the same outcome probabilities (to $< 10^{-6}$) and the same final fidelities.
- **Independent method at $L = 3$–9.** A separate implementation of the Pauli-proxy method of Takada, Bartlett and Williamson agrees on 14 400 trajectories (about 5.3×10^6 measurements, 4.2×10^5 of them random) exactly.
- **Negative control.** Removing the Pauli dressing (the term $q(b + s + x) + q(b + s)$ in Eq. (4)) breaks the agreement within the first trajectories.

7 Why it applies to study 1, and why we should use it

7.1 The assumptions hold

assumption of the Proposition	situation in study 1
the initial state is an error-free code state (so Eq. (3) holds)	the memory experiment starts from the code state
the noise consists of Pauli errors	true for the phenomenological, code-capacity and circuit-level Pauli models
only Pauli operators, CZ gates, Z-type checks and dressed checks (possibly times Z) occur	true for the memory experiment; in the circuit-level model a Pauli fault propagated through the CNOT/CCZ circuit becomes a Pauli times CZ gates, and a faulty measurement becomes a projection followed by Pauli and CZ errors (checked against the fault tables)
the state stays on flat configurations	X errors are kept in x, so B is always flat
the structure of the $\mathbb{D}_4$ code (quadratic CZ part, check shifts inside C_0)	it is the $\mathbb{D}_4$ code

7.2 Reasons to use it

1. **No approximation.** The re-randomisation by non-commuting checks and the CZ-type residues, which the previous surrogate could not represent, are included automatically. The conclusions of study 1 (crossings near 2%, below the earlier 2.4%) rely on this.

2. **Speed.** About 6 ms per shot at $L = 9$, so the production run (10^5 shots $\times$ 10 points $\times$ 4 sizes $\times$ 3 variants) fits into 10–15 hours on one Colab machine, and $L = 11$ is within reach. The dense state vector stops at $L = 3$, and the general Pauli-proxy method is several to tens of times slower for the same task.

3. **Exact numbers, not only samples.** Whether each outcome was random and the final logical fidelity (including values strictly between 0 and 1) are available exactly, which also yields precise diagnostics such as the fraction of random outcomes in flux-free layers.

4. **An independent cross-check.** It shares no state representation with the Pauli-proxy method used in studies 2 and 3, so their agreement is strong evidence for both.

7.3 When not to use it

- Protocols with a logical non-Clifford gate such as the switching unitary U (studies 2 and 3 use the Pauli-proxy method).

- Patches with boundaries: the principle is the same, but the implementation is for the torus.

- Non-Pauli (coherent) noise, or faults inside a T-gate decomposition of the CCZ gate.

- Other codes: the method relies on flat configurations and a cubic sign. At higher levels of the hierarchy (study 5, $N \geq 4$) the checks contain CCZ gates and the form is not preserved.

8 Relation to the method of Takada, Bartlett and Williamson

Their Clifford-stabilizer simulation tracks groups of stabilizers that may not commute as operators but commute on the stabilized subspace. The twisted frame is a *special case*:

- the commutator of two dressed checks is a product of Z-type checks, so each measurement is effectively commuting or anticommuting and its outcome is certain or a fair coin (their Sec. VI.A);
- case (ii) is their update rule for anticommuting measurements (Secs. IV and VI.B), written for the state;
- the term $q(b + s + x) + q(b + s)$ is their Pauli dressing.

The difference is what is stored. Their method keeps a Pauli-stabilizer tableau of a proxy state in which non-Pauli factors are replaced by Pauli ones; it applies to any code that meets its conditions. The twisted frame uses the structure of the $\mathbb{D}_4$ code to store the support and signs of the true state directly; it is specific to this code, gives amplitude information directly, and is faster.

The name *twisted frame* is not an established term but was introduced in this project; in the paper it should be introduced as "we call this representation the *twisted frame*".